

Reviewer Pack — Minimal Order of Tropical Hodge Decomposition and Resolution...

Entry b2fa14c7fd63 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 22:48:28 UTC

Minimal Order of Tropical Hodge Decomposition and Resolution Proof Width

Entry ID: b2fa14c7fd63 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 22:48:28 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ associated with a d -regular graph G , the minimal order of the tropical Hodge decomposition of its associated tropical variety is linearly correlated with its resolution proof width $w(\varphi)$, such that the order of the Hodge decomposition $\text{OHD}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Tropical geometry provides a combinatorial interpretation of algebraic objects, and its Hodge structure could potentially expose hidden complexities in the resolution process. A higher order in the tropical Hodge decomposition may indicate a more complex interaction between variables, which could correlate with the proof width.

Taxonomy category: TROPICALGEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 54fef189b8546236

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the 30 random seeds, the correlation coefficient between the order of the tropical Hodge decomposition $\text{OHD}(\varphi_G)$ and the resolution proof width $w(\varphi_G)$ is ≥ 0.8 , with all $\text{OHD}(\varphi_G) \leq 3w(\varphi_G)$, and not falsified if any seed produces an $\text{OHD}(\varphi_G) > 3w(\varphi_G)$

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'resolution proof complexity' AND 'Hodge decomposition' - 'Tseitin formula' AND 'd-regular graph' AND 'minimal order' AND 'tropical variety' - 'proof width' AND ' $\Theta(w(\phi))$ ' AND 'tropical Hodge decomposition'

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/1306.3497v2] The number of vertices of a tropical curve is bounded by its area - [http://arxiv.org/abs/hep-th/9906084v5] The mass formula for a fundamental string as a BPS solution of a D-brane's worldvolume - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2407.17951v1] Pruning Boolean d-DNNF Circuits Through Tseitin-Awareness - [http://arxiv.org/abs/2508.18083v2] GWTC-4.0: Population Properties of Merging Compact Binaries - [http://arxiv.org/abs/2112.13219v5] Measurement of $e^+e^- \rightarrow \varphi\pi^+\pi^-$ cross sections at center-of-mass energies from 2.00 to 3.08 GeV

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import itertools

def generate_random_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = [[] for _ in range(n)]
    edges = set()

    while len(edges) < (n * d) // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            graph[u].append(v)
            graph[v].append(u)
            edges.add((u, v))

    return graph

def generate_tseitin_formula(n, d):
    graph = generate_random_graph(n, d)
```

```

literals = list(range(1, n * 2 + 1))
clauses = []

for i in range(n):
    clauses.append([literals[i], -literals[n + i]])
    for j in range(i + 1, n):
        clauses.append([-literals[i], -literals[j], literals[n + i + j]])
        clauses.append([-literals[j], -literals[i], literals[n + i + j]])

return graph, literals, clauses

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i
        for j in range(i + 1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        pivot = Fraction(matrix[i][i])
        for j in range(cols):
            matrix[i][j] /= pivot

        for j in range(rows):
            if j != i:
                factor = Fraction(matrix[j][i])
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]

    return matrix

def resolution_width(clauses):
    queue = clauses[:]
    learned_clauses = []
    while queue:
        clause1 = queue.pop(0)
        for clause2 in queue + learned_clauses:
            if not set(clause1).isdisjoint(set(clause2)):
                new_clause = [l for l in clause1 if l not in clause2] + [l for l in clause2 if -l
                    in clause1]
                if len(new_clause) == 0:
                    return math.inf
                if new_clause not in queue and new_clause not in learned_clauses:
                    learned_clauses.append(new_clause)
    return max(len(c) for c in learned_clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    d = 2
    graph, literals, clauses = generate_tseitin_formula(n, d)

    OHD = len(gaussian_elimination([Fraction(1 if j in neighbors else -1) for j in range(n)] for
    w = resolution_width(clauses)

```

```

return {
    "metric_name": "OHD vs. Resolution Width",
    "metric_value": OHD / w,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": OHD <= 3 * w,
    "counterexample": "" if OHD <= 3 * w else f"OHD({OHD}) > 3w({3 * w})"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00dd1b0b.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00dd1b0b.py", line 87, in run_trial
    graph, literals, clauses = generate_tseitin_formula(n, d)
                                ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00dd1b0b.py", line 42, in generate_tseitin_formula
    clauses.append([-literals[i], -literals[j], literals[n + i + j]])
                    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17684
2	propose	ollama_remote	glm4:latest	0	0	12208
3	preregistration	ollama_remote	glm4:latest	0	0	14055
4	novelty	ollama_remote	glm4:latest	0	0	8637
5	novelty	ollama_remote	glm4:latest	0	0	10273
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15218
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39564
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18729
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14419
10	judge	ollama_remote	glm4:latest	0	0	95359

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 246145 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/b2fa14c7fd63.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b2fa14c7fd63.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b2fa14c7fd63.tar.gz (if generated)